

```
1  #include <cs50.h>
2  #include <stdio.h>
3
4  typedef struct
5  {
6      string name;
7      int votes;
8  } candidate;
9
10 candidate get_candidate(void);
11
12 int main(void)
13 {
14     candidate candidates[3];
15
16     for (int i = 0; i < 3; i++)
17     {
18         candidates[i] = get_candidate();
19     }
20
21     for (int i = 0; i < 3; i++)
22     {
23         printf("Candidate %i is named %s and has %i votes.\n", i + 1, candidates[i].name,
24             candidates[i].votes);
25     }
26 }
27
28 // Function to get a new candidate
29 candidate get_candidate(void)
30 {
31     candidate new_candidate;
32     new_candidate.name = get_string("Name: ");
33     new_candidate.votes = get_int("Votes: ");
34
35     return new_candidate;
36 }
37
```

```
1  #include <cs50.h>
2  #include <stdio.h>
3
4  int factorial(int n);
5
6  int main(void)
7  {
8      int n = get_int("Factorial of: ");
9      printf("Factorial of %i is %i\n", n, factorial(n));
10 }
11
12 int factorial(int n)
13 {
14     // Base case
15     if (n == 0)
16     {
17         return 1;
18     }
19
20     // Recursive case
21     return n * factorial(n - 1);
22 }
```

```
1  #include <cs50.h>
2  #include <stdio.h>
3
4  int fib(int n);
5
6  int main(void)
7  {
8      int n = get_int("Fibonacci number: ");
9      printf("The %i Fibonacci number is %i\n", n, fib(n));
10 }
11
12 int fib(int n)
13 {
14     // Base Case
15     if (n == 0)
16     {
17         return 0;
18     }
19     if (n == 1)
20     {
21         return 1;
22     }
23
24     // Recursive Case
25     return fib(n - 1) + fib(n - 2);
26 }
```